📑 Basic Information:

Experiment No: 13

Date:

Title: Generating Phone Keypad Letter Combinations Using Recursion in Java.

🎯 Objective /Aim:

To write a Java program that generates all possible letter combinations represented by a string of digits, using the letter mappings of a phone keypad.

Given a string containing digits from 2-9 inclusive, return all possible letter combinations that the number could represent.

A mapping of digit to letters (just like on the telephone buttons) is given below. Note that 1-does not map to any letters.

Example: Input: "23"

🛠️ System Requirements:

Hardware:

-> Standard PC/Laptop

-> Intel Core i3/i5 processor or equivalent

-> Minimum 4 GB RAM

-> Minimum 500 MB free

Software:

->Operating System: Windows / Linux / macOS

-> Java Development Kit (JDK 17 or JDK 21)

-> Text Editor or IDE such as Eclipse, IntelliJ IDEA, NetBeans, or VS Code

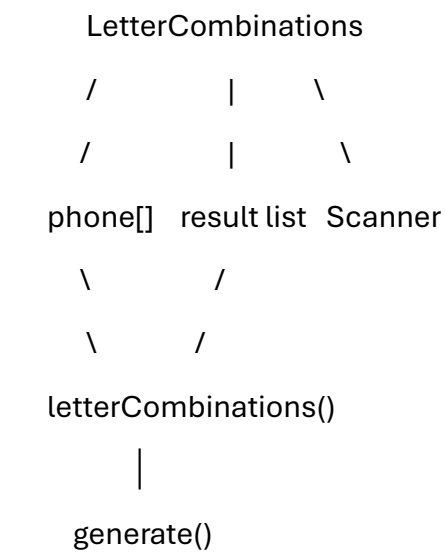
-> Command Prompt / Terminal

Algorithm/Logic Description:

1. Start the program.
2. Store the phone keypad letter mapping for digits 0 through 9.
3. Read a string of digits from the user.
4. Clear the list of previous combinations.
5. If the input is empty, return the empty list.

6. Call the recursive generate method with the digit string, starting index 0, and an empty current combination.
7. In generate:
    - If the index reaches the end of the digit string, add the current combination to the result list.
    - Otherwise, get the letters mapped to the current digit.
    - For each mapped letter, append it to the current combination and recursively process the next digit.
8. Display the list of combinations.
9. Close the scanner and stop the program.

UML Class Diagram:



Source Code:

```
import java.util.*;

// Generates letter combinations for a string of phone keypad digits

public class LetterCombinations {

  // Maps each digit to its corresponding phone keypad letters

  static String[] phone = {

    "", "", "abc", "def", "ghi",

    "jkl", "mno", "pqrs", "tuv", "wxyz"

  };

  // Stores all generated combinations
```

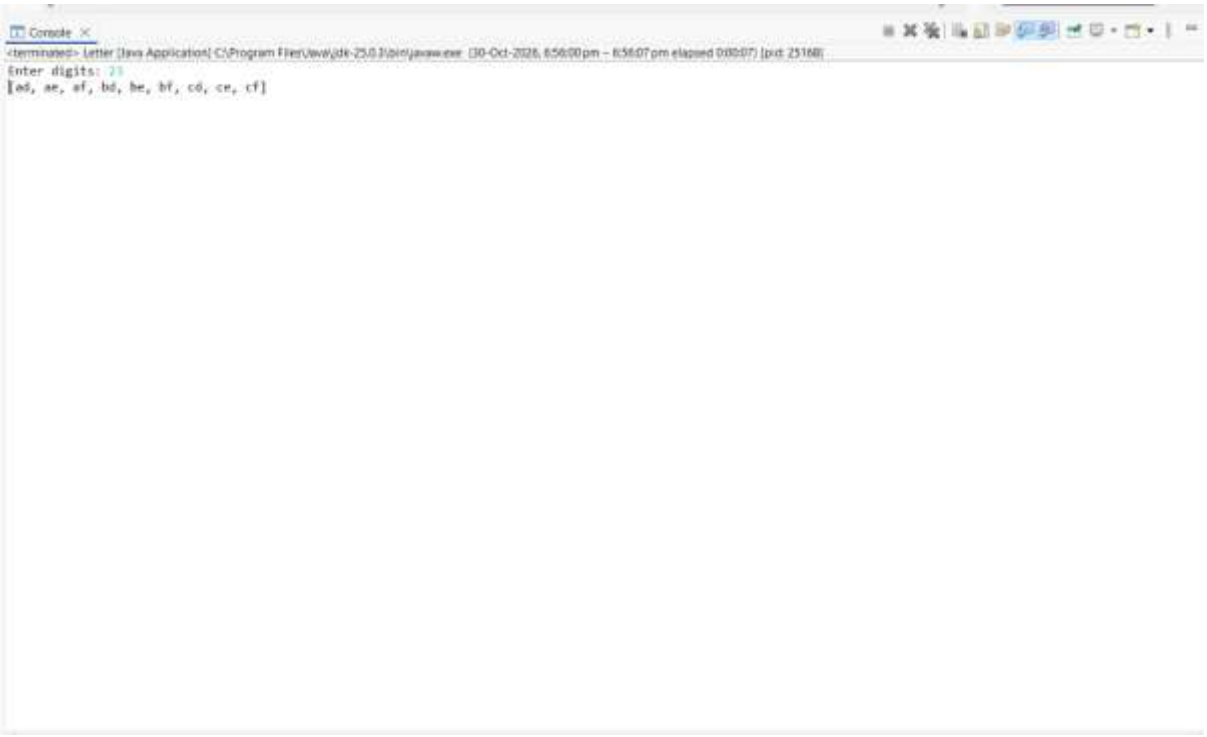
```
static List<String> result = new ArrayList<>();
// Returns all possible letter combinations for the given digits
public static List<String> letterCombinations(String digits) {
   // Clear previous results before generating new combinations
   result.clear();
   // Return an empty list if the input is null or empty
   if (digits == null || digits.length() == 0) {
      return result;
   }
   // Start generating combinations from the first digit
   generate(digits, 0, "");
   return result;
}
// Recursively builds combinations one digit at a time
static void generate(String digits, int index, String current) {
   // If all digits have been processed, save this combination
   if (index == digits.length()) {
      result.add(current);
      return;
   }
   // Convert the current digit character to an integer
   int digit = digits.charAt(index) - '0';
   // Try each letter mapped to this digit
   for (int i = 0; i < phone[digit].length(); i++) {
      char ch = phone[digit].charAt(i);
      // Add the letter and process the next digit
      generate(digits, index + 1, current + ch);
   }
}
```

```
public static void main(String[] args) {
        // Read digits from the user
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter digits: ");
        String digits = sc.nextLine();
        // Generate and display all possible combinations
        System.out.println(letterCombinations(digits));
        // Close the scanner
        sc.close();
    }
}
```

Relationship:

The LetterCombinations class uses the phone array to map digits to letters and the result list to store combinations. Its letterCombinations() method calls generate(), which recursively builds each combination. The main() method reads the digits and displays the results.

Output Screenshot:



Result: The program generates and displays all possible letter combinations corresponding to the digits entered, using the phone keypad mapping and recursion.